РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Кафедра теории вероятностей и математической статистики

ОТЧЁТ

по лабораторной работе № 5

«Расширенная настройка HTTP-сервера Apache»

Дисциплина: Администрирование сетевых подсистем

Выполнил: студент группы _____

Колабов Д.С.

Проверил: Королькова А.В.

Москва, 2026

1. Задание

Цель работы: приобретение практических навыков по расширенному конфигурированию HTTP-сервера Apache в части безопасности и возможности использования PHP.

В ходе работы необходимо:

1. Сгенерировать криптографический ключ и самоподписанный сертификат безопасности для обеспечения работы веб-сервера по протоколу HTTPS.
2. Настроить веб-сервер для работы с PHP.
3. Скорректировать скрипт для Vagrant, фиксирующий действия по расширенной настройке HTTP-сервера во внутреннем окружении виртуальной машины server.

2. Описание результатов выполнения задания

2.1. Конфигурирование HTTP-сервера для работы через протокол HTTPS

На виртуальной машине server в каталоге /etc/pki/tls/private сгенерированы закрытый ключ и самоподписанный SSL-сертификат:

```
mkdir -p /etc/pki/tls/private
ln -s /etc/pki/tls/private /etc/ssl/private
cd /etc/pki/tls/private
openssl req -x509 -nodes -newkey rsa:2048 -keyout
www.kolade.net.key -out www.kolade.net.crt
mv www.kolade.net.crt /etc/pki/tls/certs
```

Параметры команды openssl:

- req -x509 — запрос подписи сертификата в формате X.509 (стандарт инфраструктуры открытых ключей);
- -nodes — пропустить шифрование закрытого ключа парольной фразой, чтобы Apache читал файл без ввода пароля;
- -newkey rsa:2048 — одновременно создаётся новый ключ и сертификат; используется 2048-битный ключ RSA;
- -keyout — путь для сохранения файла закрытого ключа;
- -out — путь для сохранения файла сертификата SSL.

Данные, введённые при заполнении сертификата: Country = RU, State = Russia, Locality = Moscow, Organization = kolade, Organizational Unit = kolade, Common Name = kolade.net, Email = kolade@kolade.net.

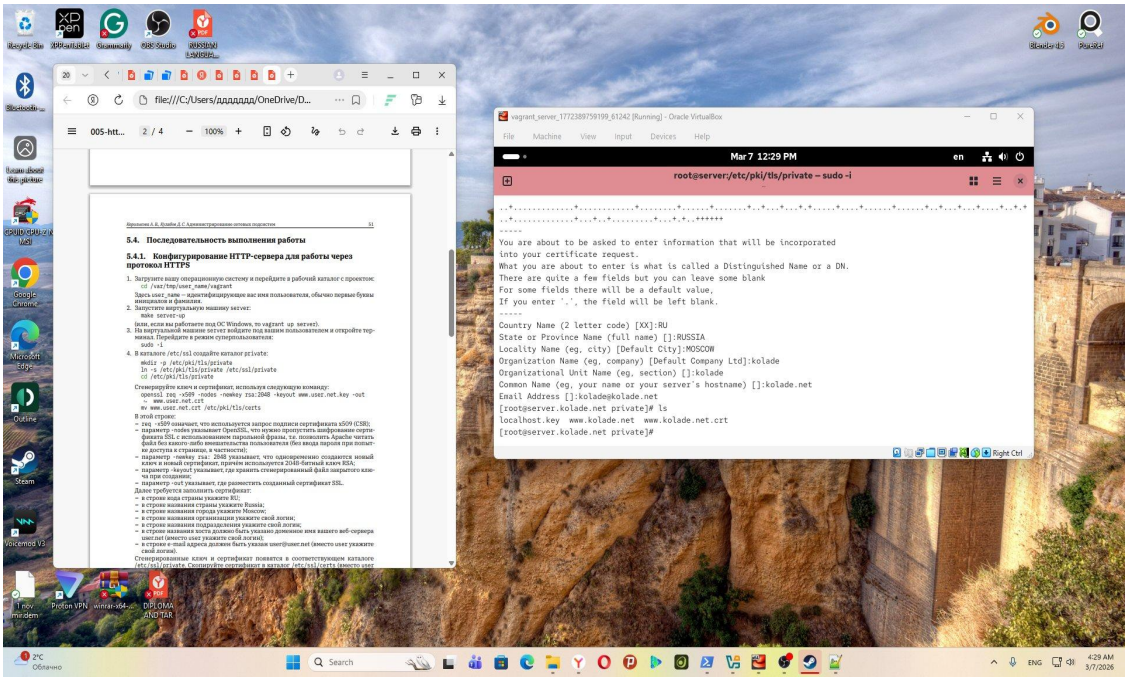

Рис. 1. Генерация SSL-ключа и сертификата (первый сеанс, 12:29)

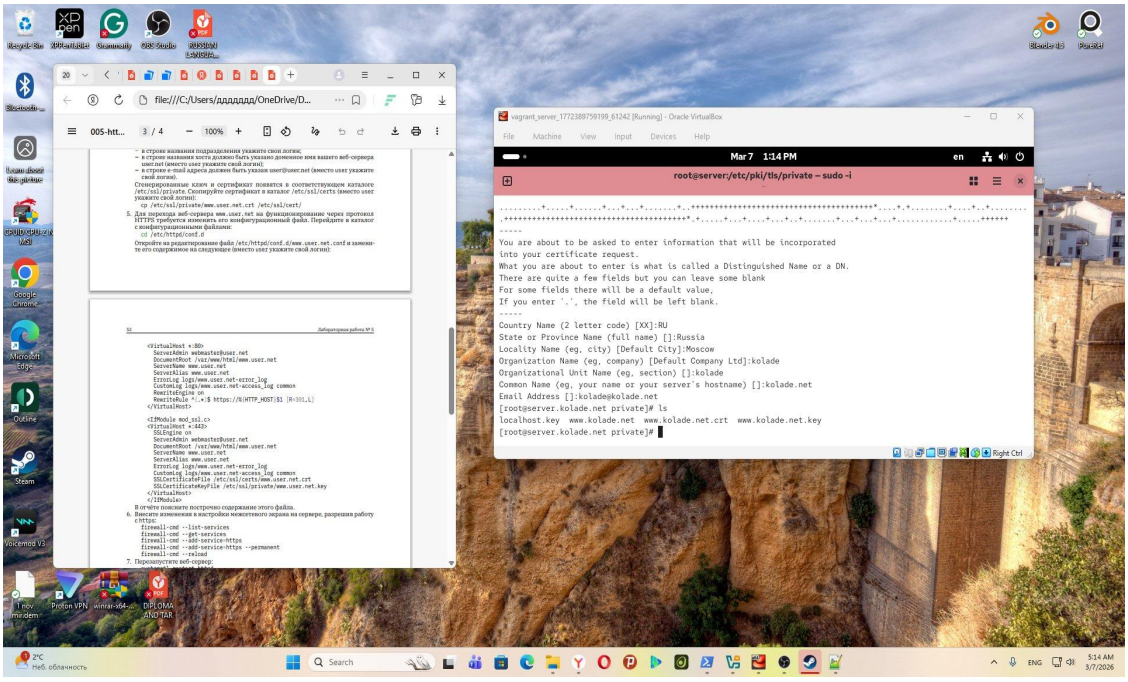

Рис. 2. Генерация SSL-ключа и сертификата для www.kolade.net (5:14, второй сертификат)

Конфигурационный файл /etc/httpd/conf.d/www.kolade.net.conf изменён для поддержки HTTPS. Полное содержимое файла:

```
<VirtualHost *:80>
    ServerAdmin webmaster@kolade.net
    DocumentRoot /var/www/html/www.kolade.net
```

```
    ServerName www.kolade.net
    ServerAlias www.kolade.net
    ErrorLog logs/www.kolade.net-error_log
    CustomLog logs/www.kolade.net-access_log common
    RewriteEngine on
    RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L]
</VirtualHost>

<IfModule mod_ssl.c>
<VirtualHost *:443>
    SSLEngine on
    ServerAdmin webmaster@kolade.net
    DocumentRoot /var/www/html/www.kolade.net
    ServerName www.kolade.net
    ServerAlias www.kolade.net
    ErrorLog logs/www.kolade.net-error_log
    CustomLog logs/www.kolade.net-access_log common
    SSLCertificateFile /etc/ssl/certs/www.kolade.net.crt
    SSLCertificateKeyFile /etc/ssl/private/www.kolade.net.key
</VirtualHost>
</IfModule>
```

Построчное пояснение конфигурационного файла:

- <VirtualHost *:80> — блок виртуального хоста для HTTP (порт 80), принимающий подключения со всех IP-адресов.
- ServerAdmin — адрес электронной почты администратора сервера для уведомлений об ошибках.
- DocumentRoot — корневой каталог, из которого Apache обслуживает файлы сайта.
- ServerName — основное доменное имя виртуального хоста.
- ServerAlias — псевдоним (дополнительное имя), по которому также доступен хост.
- ErrorLog / CustomLog — пути к журналам ошибок и доступа соответственно.
- RewriteEngine on — активирует модуль mod_rewrite для обработки правил переписывания URL.
- RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L] — перенаправляет все HTTP-запросы на HTTPS с кодом 301 (постоянное перенаправление); [L] — это последнее правило в наборе.
- <IfModule mod_ssl.c> — условный блок, активируемый только при загруженном модуле mod_ssl.
- <VirtualHost *:443> — блок виртуального хоста для HTTPS (порт 443).
- SSLEngine on — включает SSL/TLS шифрование для данного виртуального хоста.

- SSLCertificateFile — путь к файлу публичного сертификата, предъявляемого клиентам.
- SSLCertificateKeyFile — путь к файлу закрытого ключа для дешифрования соединения.

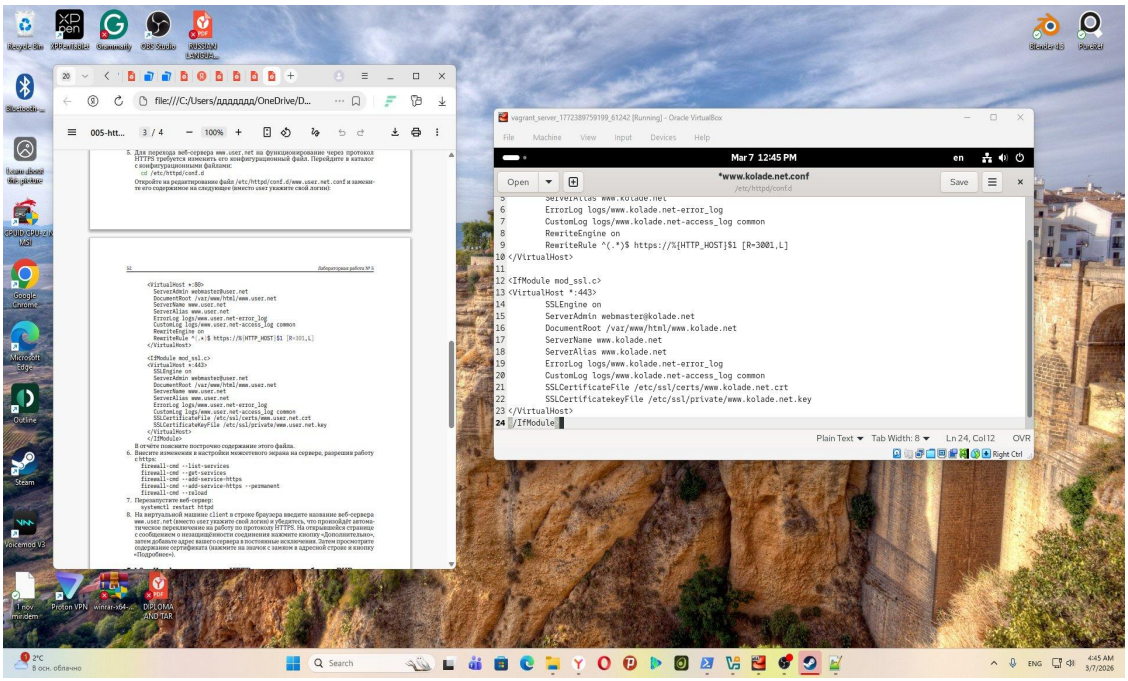

Рис. 3. Конфигурационный файл www.kolade.net.conf с настройками HTTPS в редакторе

В межсетевом экране сервера разрешена работа с HTTPS:

```
firewall-cmd --list-services
firewall-cmd --get-services
firewall-cmd --add-service=https
firewall-cmd --add-service=https --permanent
firewall-cmd --reload
```

Веб-сервер перезапущен командой systemctl restart httpd.

Проверка с виртуальной машины client: при обращении к www.kolade.net в браузере произошло автоматическое перенаправление на HTTPS. Самоподписанный сертификат добавлен в исключения браузера и успешно просмотрен:

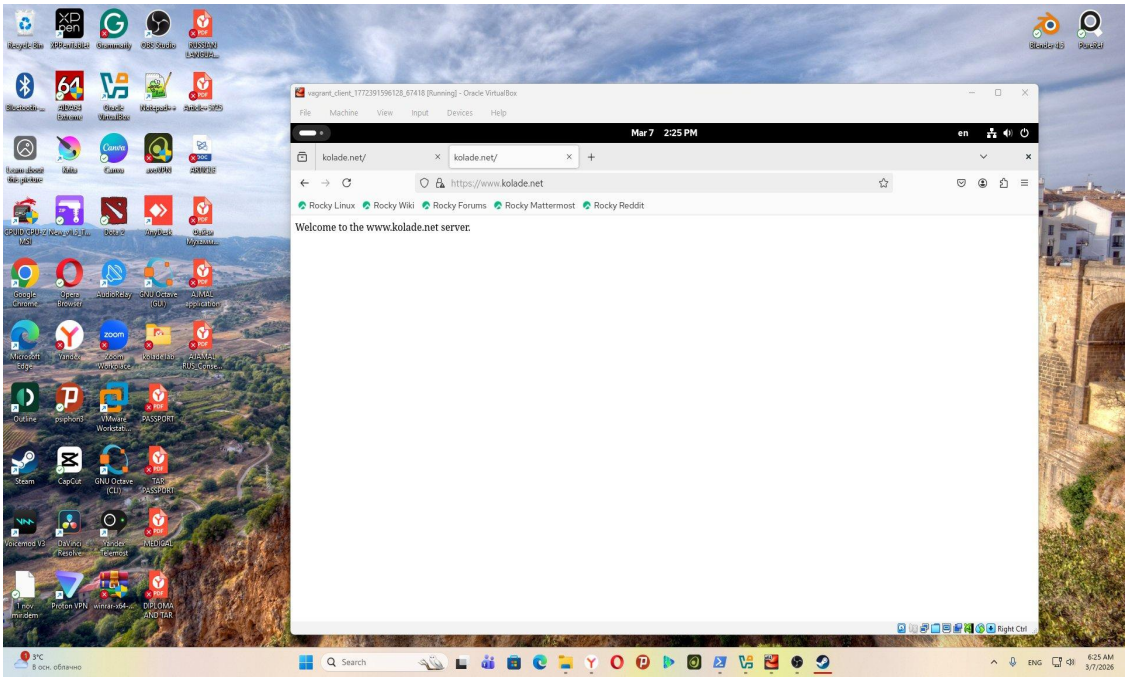

Рис. 4. Страница www.kolade.net открыта по HTTPS, редирект с HTTP выполнен

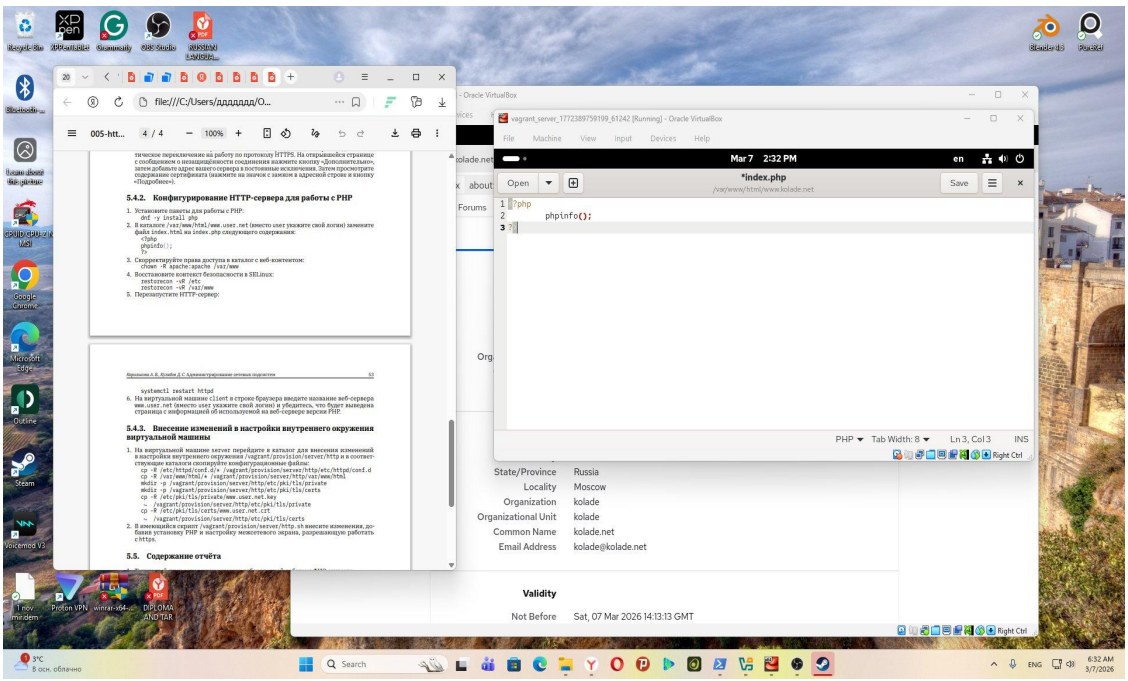

Рис. 5. Просмотр SSL-сертификата: Subject Name — kolade.net, RU, Moscow, kolade

2.2. Конфигурирование HTTP-сервера для работы с PHP

Установлен пакет PHP:

```
dnf -y install php
```

В каталоге /var/www/html/www.kolade.net файл index.html заменён на index.php:

```
<?php
    phpinfo();
?>
```

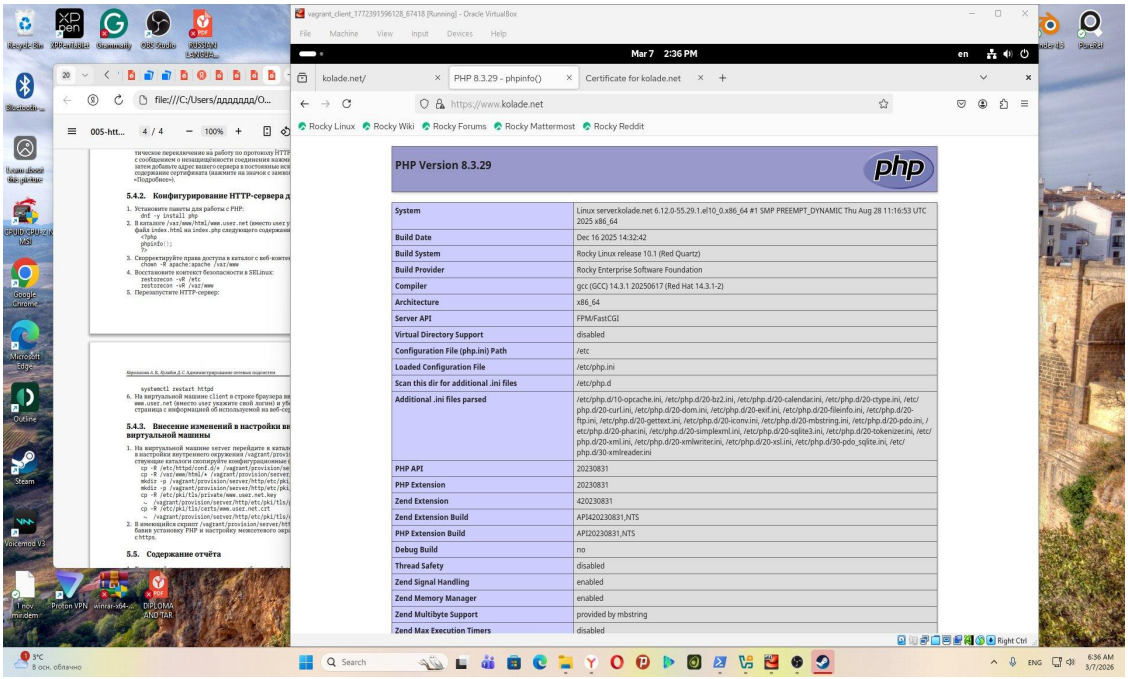

Рис. 6. Файл index.php с вызовом phpinfo() открыт в редакторе на сервере

Скорректированы права доступа к веб-контенту и восстановлен контекст безопасности SELinux:

```
chown -R apache:apache /var/www
restorecon -vR /etc
restorecon -vR /var/www
```

После перезапуска HTTP-сервера (systemctl restart httpd) проверка с виртуальной машины client показала корректную работу PHP: при открытии https://www.kolade.net отображена страница phpinfo() с версией PHP 8.3.29:

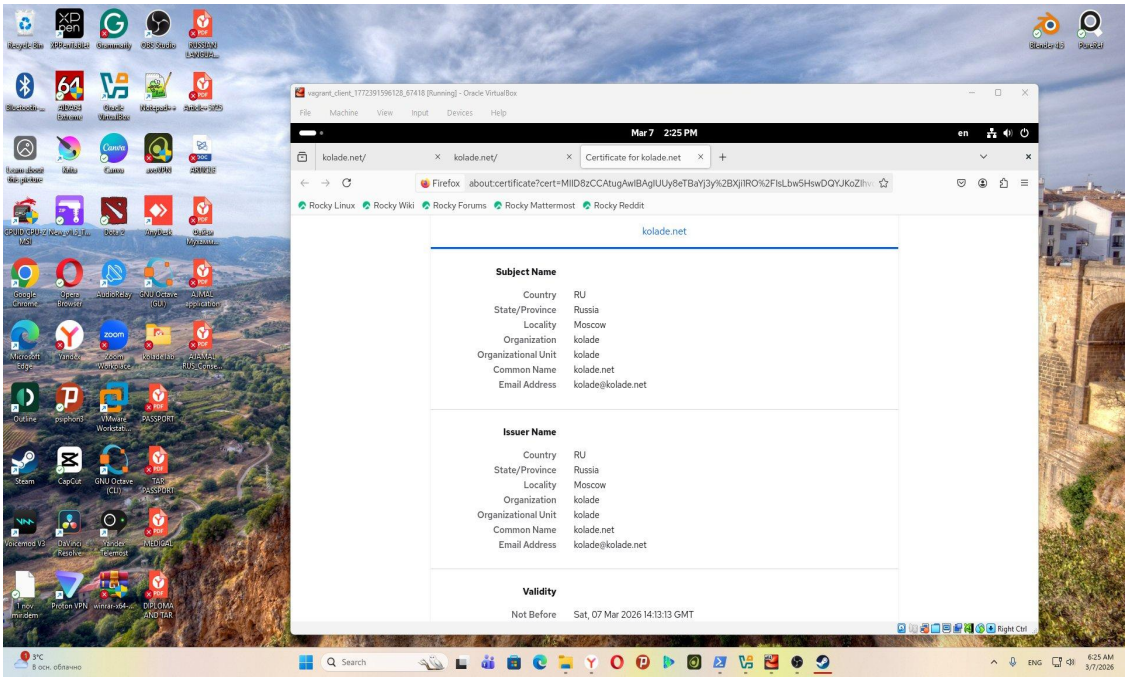

Рис. 7. Страница phpinfo() — PHP Version 8.3.29 на Rocky Linux 10.1

2.3. Внесение изменений в настройки внутреннего окружения виртуальной машины

Конфигурационные файлы и сертификаты скопированы в каталог Vagrant:

```
cp -R /etc/httpd/conf.d/*
/vagrant/provision/server/http/etc/httpd/conf.d
cp -R /var/www/html/* /vagrant/provision/server/http/var/www/html
mkdir -p /vagrant/provision/server/http/etc/pki/tls/private
mkdir -p /vagrant/provision/server/http/etc/pki/tls/certs
cp -R /etc/pki/tls/private/www.kolade.net.key \
    /vagrant/provision/server/http/etc/pki/tls/private
cp -R /etc/pki/tls/certs/www.kolade.net.crt \
    /vagrant/provision/server/http/etc/pki/tls/certs
```

Скрипт http.sh обновлён: добавлена строка установки PHP и правила firewall для HTTPS. Итоговое содержимое:

```
#!/bin/bash
echo "Provisioning script $0"
echo "Install needed packages"
dnf -y groupinstall "Basic Web Server"
dnf -y install php

echo "Copy configuration files"
cp -R /vagrant/provision/server/http/etc/httpd/* /etc/httpd
cp -R /vagrant/provision/server/http/var/www/* /var/www
chown -R apache:apache /var/www
restorecon -vR /etc
```

```
restorecon -vR /var/www

echo "Configure firewall"
firewall-cmd --add-service=http
firewall-cmd --add-service=http --permanent
firewall-cmd --add-service=https
firewall-cmd --add-service=https --permanent

echo "Start http service"
systemctl enable httpd
systemctl start httpd
```

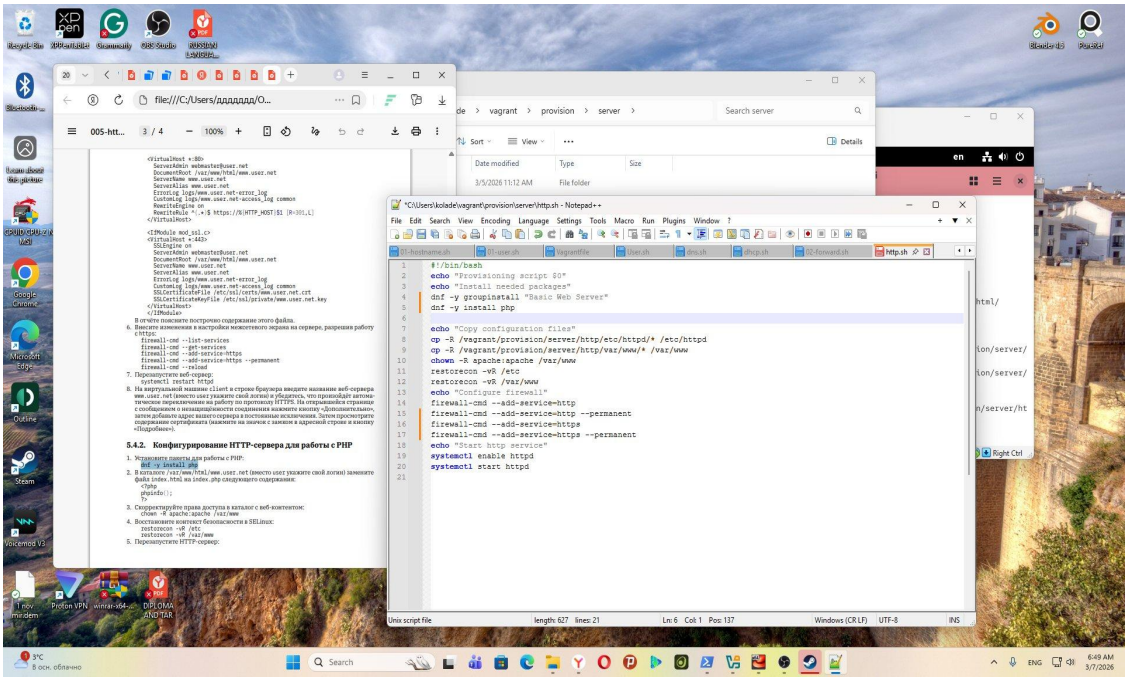

Рис. 8. Итоговый скрипт http.sh с поддержкой PHP и HTTPS

3. Выводы

В ходе лабораторной работы получены практические навыки по расширенному конфигурированию HTTP-сервера Apache:

- Сгенерирован самоподписанный SSL-сертификат и закрытый RSA-2048 ключ с помощью утилиты openssl. Сертификат содержит информацию об организации, домене и контактах администратора.
- Веб-сервер www.kolade.net переведён на HTTPS: настроено автоматическое перенаправление с HTTP на HTTPS (код 301), подключены сертификат и ключ в блоке VirtualHost *:443, разрешён порт HTTPS в межсетевом экране.

- Корректность работы HTTPS подтверждена: браузер клиента автоматически перенаправляется на защищённое соединение, сертификат успешно отображён.
- Установлен и проверен PHP 8.3.29: страница phpinfo() успешно открывается по HTTPS, что подтверждает работу PHP как серверного языка через Apache.
- Скрипт http.sh для Vagrant обновлён: добавлены установка PHP и правила firewall для HTTPS, что обеспечивает автоматическое воспроизведение всей конфигурации при развёртывании виртуальной машины.

Применение протокола HTTPS критически важно для защиты передаваемых данных. PHP расширяет возможности веб-сервера, позволяя создавать динамические веб-приложения.

4. Ответы на контрольные вопросы

1. В чём отличие HTTP от HTTPS?

HTTP (HyperText Transfer Protocol) — протокол передачи гипертекста, передающий данные в открытом (нешифрованном) виде. Это делает его уязвимым для атак типа «человек посередине» (MITM): злоумышленник может перехватить или подменить данные.

HTTPS (HTTP Secure) — расширение HTTP с использованием протоколов SSL/TLS для шифрования. Ключевые отличия:

- Данные шифруются, что исключает возможность их чтения при перехвате.
- HTTP использует порт 80, HTTPS — порт 443.
- HTTPS требует наличия сертификата SSL/TLS, подтверждающего подлинность сервера.
- HTTPS обеспечивает аутентификацию сервера: клиент убеждается в подлинности ресурса.
- HTTPS защищает целостность данных: исключает их модификацию при передаче.

2. Каким образом обеспечивается безопасность контента веб-сервера при работе через HTTPS?

Безопасность при HTTPS обеспечивается тремя основными механизмами:

- Асимметричное шифрование (TLS Handshake): при установке соединения сервер предъявляет сертификат с открытым ключом. Клиент проверяет его подлинность и с помощью асимметричного шифрования безопасно согласовывает сеансовый ключ, не передавая его в открытом виде по сети.

- Симметричное шифрование данных: после установки соединения весь трафик шифруется симметричным алгоритмом (например, AES-256) с использованием согласованного сеансового ключа. Это обеспечивает конфиденциальность данных.
- Коды аутентификации сообщений (MAC / HMAC): обеспечивают целостность передаваемых данных. Любая модификация сообщения в процессе передачи будет обнаружена получателем.

Сертификат SSL/TLS, подписанный доверенным центром сертификации (CA), гарантирует подлинность сервера и исключает его подмену злоумышленником.

3. Что такое сертификационный центр? Приведите пример.

Сертификационный центр (Certification Authority, CA) — это доверенная организация в инфраструктуре открытых ключей (PKI), которая выдаёт и подписывает цифровые сертификаты. CA своей подписью подтверждает, что открытый ключ из сертификата действительно принадлежит указанному владельцу (лицу, организации или серверу).

Принцип работы CA:

- Владелец сервера генерирует пару ключей (открытый и закрытый) и отправляет CA запрос на подпись сертификата (CSR) с открытым ключом.
- CA верифицирует личность/домен заявителя и подписывает сертификат своим закрытым ключом.
- Браузеры и ОС содержат встроенный список доверенных корневых CA. Если цепочка сертификатов ведёт к доверенному CA, браузер принимает соединение без предупреждений.

Примеры сертификационных центров:

- Let's Encrypt — некоммерческий CA, предоставляющий бесплатные автоматически обновляемые сертификаты. Наиболее популярен для веб-серверов в интернете.
- DigiCert — один из крупнейших коммерческих CA, выдаёт сертификаты для корпоративных клиентов.
- GlobalSign, Comodo (Sectigo), GeoTrust — другие известные коммерческие CA.

В данной лабораторной работе использован самоподписанный сертификат (self-signed certificate) — выдан и подписан самим сервером, без участия стороннего CA. Браузер выдаёт предупреждение о незащищённом соединении, однако для учебных целей и внутреннего использования такой сертификат вполне подходит.